

INTRODUCTION TO PROGRAMMING

UNIT-2

Control Structures

Simple sequential programs Conditional Statements (if, if-else, switch), Loops (for, while, do while) Break and Continue.

The control statements help users specify the order of execution of the instructions present in a program.

These make it possible for the program to make certain decisions, perform various tasks repeatedly, or even jump from any one section of the code to a different section. Control statements serve to control the execution flow of a program.

Control statements in C are classified into selection statements, iteration statements, and jump statements. Selection statements serve to execute code based on a certain circumstance. Iteration statements loop through a code block until a condition is met. Jump statements serve to move control from one section of a program to another.

The if statement, for loop, while loop, switch statement, break statement, and continue statement are C's most widely used control statements.

TYPES OF CONTROL STATEMENTS:

- Conditional control Statements
- Looping control statements
- Unconditional control statements

Selection Statement/Conditional Statements/Decision Making Statements:

A selection statement selects among a set of statements depending on the value of a controlling expression.

Or

Moving execution control from one place/line to another line based on condition

Or

Conditional statements control the sequence of statement execution, depending on the value of a integer expression

C language supports two conditional statements.

1: if

2: switch.

1: if Statement: The if Statement may be implemented in different forms.

1: simple if statement.

2: if –else statement

3: nested if-else statement.

4: else if ladder.

1: simple if statement:

Syntax: if (condition/expression)

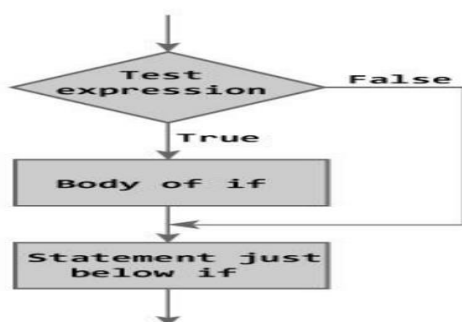
```
{  
    True statement;  
}  
  
Statement-x;
```

Working:

If the condition/expression is true, then the true statement will be executed otherwise the true Statement block will be skipped and the execution will jump to the statement-x. The “true statement” may be a single statement or group of statement.

If there is only one statement in the if block, then the braces are optional. But if there is more than one statement the braces are compulsory

Flowchart:



Example 1:

```
#include<stdio.h>

int main()
{
    int number;
    printf("Enter a number:");
    scanf("%d",&number);
    if(number%2==0){
        printf("%d is even number",number);
    }
    return 0;
}
```

Output:

Enter a number: 4

4 is even number

Example 2: Program to find the largest number of the three.

```
#include <stdio.h>

int main()
{
    int a, b, c;
    printf("Enter three numbers?");
    scanf("%d %d %d",&a,&b,&c);
    if(a>b && a>c)
    {
        printf("%d is largest",a);
    }
    if(b>a && b > c)
    {
        printf("%d is largest",b);
    }
    if(c>a && c>b)
    {
        printf("%d is largest",c);
    }
}
```

```
if(a == b && a == c)
{
    printf("All are equal");
}
}
```

Output

Enter three numbers?

12 23 34

34 is largest

2. If-else Statement: The if-else statement is an extension of the simple if statement. The general form is. The if...else statement executes some code if the test expression is true (nonzero) and some other code if the test expression is false (0).

Syntax:

```
if (condition)
{
    true statements;
}
else
{
    false statements;
}
```

Statement-x;

Working:

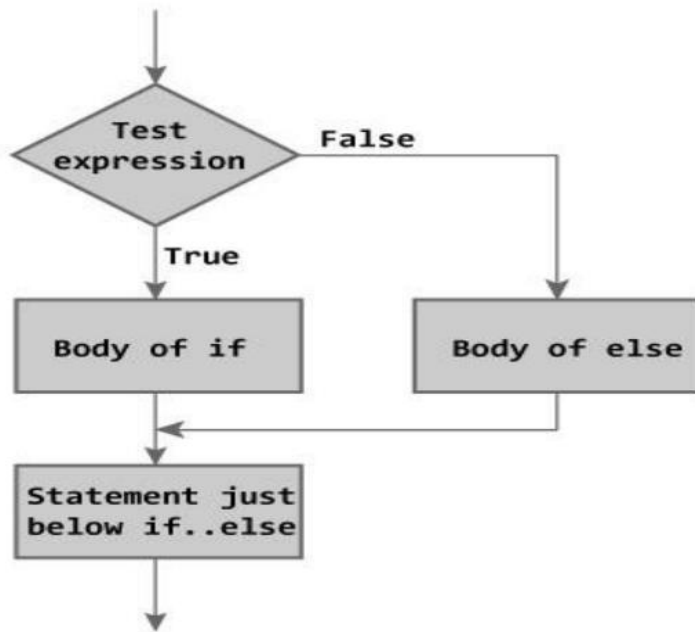
If the condition is true, then the true statement and statement-x will be executed and if the condition is false, then the false statement and statement-x is executed.

Or

If test expression is true, codes inside the body of if statement is executed and, codes inside the body of else statement is skipped.

If test expression is false, codes inside the body of else statement is executed and, codes inside the body of if statement is skipped.

Flowchart:



Example:

Program to check whether an integer entered by the user is odd or even

```
#include <stdio.h>

int main()
{
    int number;
    printf("Enter an integer: ");
    scanf("%d",&number);
    // True if remainder is 0
    if( number%2 == 0 )
        printf("%d is an even integer.",number);
    else
        printf("%d is an odd integer.",number);
    return 0;
}
```

Output:

Enter an integer: 7
7 is an odd integer.

Program to check whether a person is eligible to vote or not.

```
#include <stdio.h>

int main()
{
    int age;
    printf("Enter your age?");
    scanf("%d",&age);
    if(age>=18)
    {
        printf("You are eligible to vote...");
    }
    else
    {
        printf("Sorry ... you can't vote");
    }
}
```

Output

```
Enter your age?18
You are eligible to vote...
Enter your age?13
Sorry ... you can't vote
```

3. Nested if-else statement:

When a series of decisions are involved, we may have to use more than one if-else statement in nested form. If-else statements can also be nested inside another if block or else block or both.

Syntax:

```
if(condition-1)
{
    if (condition-2)
    {
        Statement-1;
    }
    else
    {
```

Statement-2;

}

}

else

{

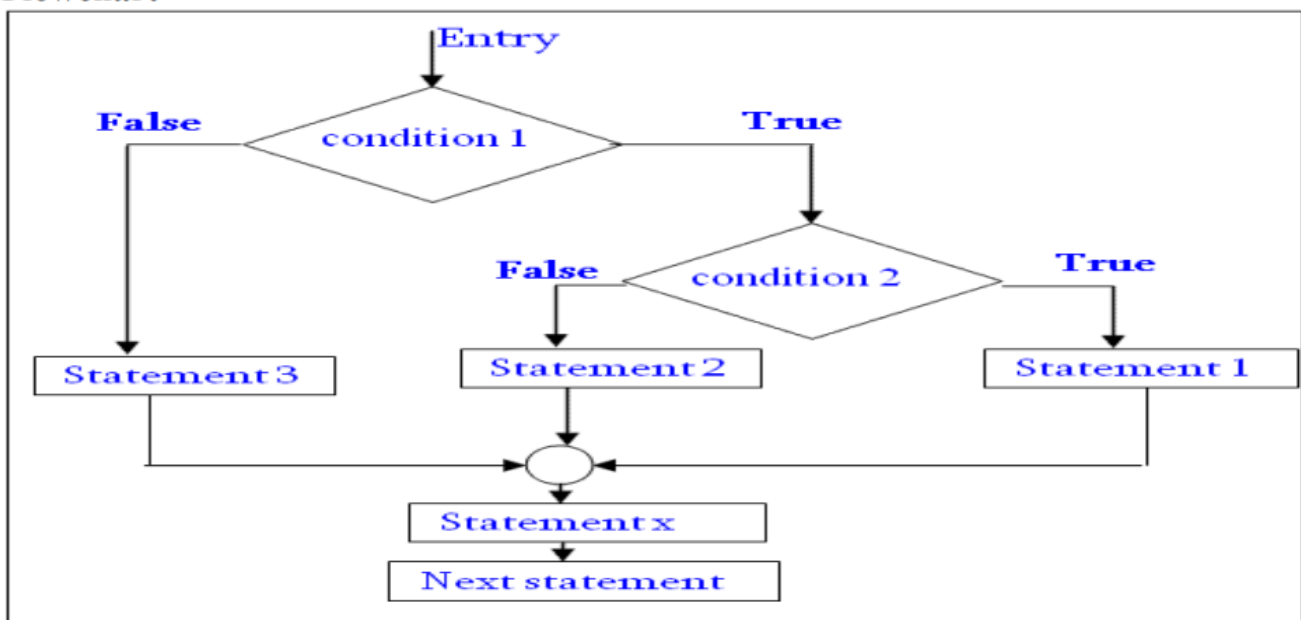
Statement-3;

}

Statement-x;

Working: If the condition-1 is false, the statement-3 and statement-x will be executed. Otherwise it continues to perform the second test. If the condition-2 is true, the true statement-1 will be executed otherwise the statement-2 will be executed and then the control is transferred to the statement-x.

Flowchart



Example

```
#include<stdio.h>
```

```
int var1, var2;
```

```
printf("Input the value of var1:");
```

```
scanf("%d", &var1);
```

```
printf("Input the value of var2:");
```

```
scanf("%d",&var2);
```

```
if (var1 !=var2)
```

```
{
```

```
printf("var1 is not equal to var2");
```

```
//Below – if-else is nested inside another if block
```

```
if (var1 > var2)
{
    printf("var1 is greater than var2");
}
else
{
    printf("var2 is greater than var1");
}
}
else
{
    printf("var1 is equal to var2");
}
```

// C program to illustrate nested-if statement

```
#include <stdio.h>

int main()
{
    int i = 10;

    if (i == 10)
    {
        // First if statement
        if (i < 15)
            printf("i is smaller than 15\n");
        if (i < 12)
            printf("i is smaller than 12 too\n");
        else
            printf("i is greater than 15");
    }
}
```



```
return 0;
```

```
}
```

Output

i is smaller than 15

i is smaller than 12 too

C program to find largest from three numbers

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int num1, num2, num3;
```

```
    printf("Enter three numbers:\n");
```

```
    scanf("%d%d%d",&num1, &num2, &num3);
```

```
    if(num1>num2)
```

```
    {
```

```
        /* This is nested if-else */
```

```
        if(num1>num3)
```

```
        {
```

```
            printf("Largest = %d", num1);
```

```
        }
```

```
        else
```

```
        {
```

```
            printf("Largest = %d", num3);
```

```
        }
```

```
    }
```

```
    else
```

```
    {
```

```
        /* This is nested if-else */  
  
        if(num2>num3)  
        {  
            printf("Largest = %d", num2);  
        }  
        else  
        {  
            printf("Largest = %d", num3);  
        }  
    }  
    return(0);  
}
```

Output:

Run 1:

```
-----  
  
Enter three numbers:  
  
12↓  
33↓  
-17↓  
  
Largest = 33
```

Run 2:

```
-----  
  
Enter three numbers:  
  
-18↓  
-31↓
```

-17↓

Largest = -17

4. if-else- if ladder:

in C programming, if-else-if statement is **also known as if-else-if ladder**. It is used when there are more than two possible action based on different conditions.

Syntax:

```
if( condition-1)
{
    Statement-1;
}
else if(condition-2)
{
    Statement-2;
}
else if(condition-3)
{
    Statement-3;
}
else if(condition-n)
{
    Statement-n;
}
else
{
    Default-statement;
```

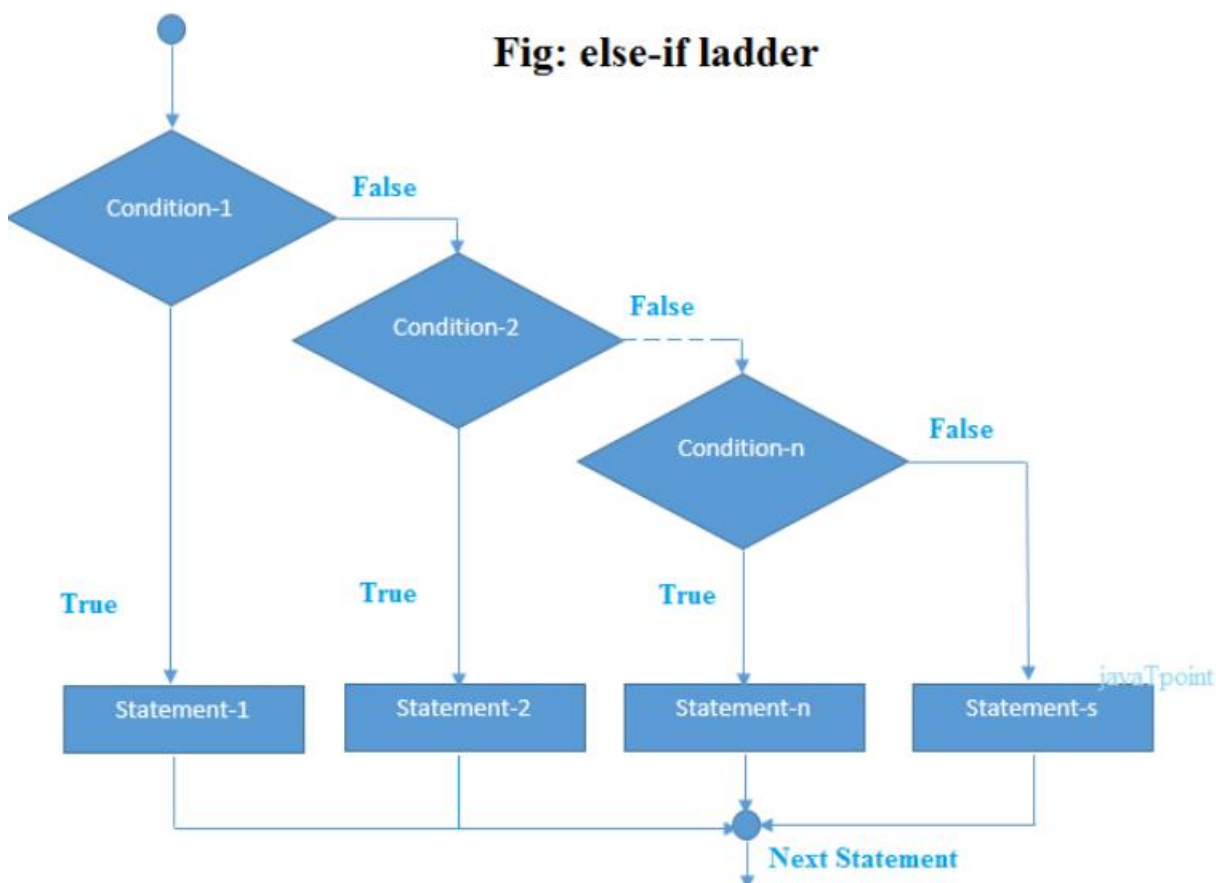
```
}
```

Statement-x;

Working:

- In the above syntax of if-else-if, if the Condition1 is TRUE then the Statement1 will be executed and control goes to next statement in the program following if-else-if ladder.
- If Condition1 is FALSE then Condition2 will be checked, if Condition2 is TRUE then Statement2 will be executed and control goes to next statement in the program following if-else-if ladder.
- Similarly, if Condition2 is FALSE then next condition will be checked and the process continues.
- If all the conditions in the if-else-if ladder are evaluated to FALSE, then Default Statement will be executed.

Flow Chart:



Example: Program to calculate the grade of the student according to the specified marks.

```
#include <stdio.h>
```

```
int main()
```

```

{
    int marks;
    printf("Enter your marks?");
    scanf("%d",&marks);
    if(marks > 85 && marks <= 100)
    {
        printf("Congrats ! you scored grade A ...");
    }
    else if (marks > 60 && marks <= 85)
    {
        printf("You scored grade B + ...");
    }
    else if (marks > 40 && marks <= 60)
    {
        printf("You scored grade B ...");
    }
    else if (marks > 30 && marks <= 40)
    {
        printf("You scored grade C ...");
    }
    else
    {
        printf("Sorry you are fail ...");
    }
}

```

Output:

Enter your marks? 10

Sorry you are fail...

Enter your marks? 40

You scored grade C...

Enter your marks? 90

Congrats! You scored grade A...

Example 2: C program to find largest from three numbers given by user to explain working of if-else-if statement or ladder

```
#include<stdio.h>
int main()
{
    int a,b,c;
    printf("Enter three numbers: \n");
    scanf("%d%d%d", &a, &b, &c);
    if(a>b && a>c)
    {
        printf("Largest = %d", a);
    }
    else if(b>a && b>c)
    {
        printf("Largest = %d", b);
    }
    else
    {
        printf("Largest = %d", c);
    }
    return(0);
}
```

Output

Run 1:

Enter three numbers:

12 ↵

33 ↵

-17 ↵

Largest = 33

Run 2:

Enter three numbers:

-18 ↵

-31 ↵

-17 ↵

Largest = -17

Example 3: C program to print weekday based on given number.

```
#include<stdio.h>

int main()
{
    int day;
    printf("Enter day number: ");
    scanf("%d", &day);
    if(day==1)
    {
        printf("SUNDAY.");
    }
    else if(day==2)
    {
        printf("MONDAY.");
    }
    else if(day==3)
    {
        printf("TUESDAY.");
    }
    else if(day==4)
    {
        printf("WEDNESDAY.");
    }
    else if(day==5)
    {
        printf("THURSDAY.");
    }
    else if(day==6)
    {
```

```

        printf("FRIDAY.");
    }
    else if(day==7)
    {
        printf("SATURDAY.");
    }
    else
    {
        printf("INVALID DAY.");
    }
    return(0);
}

```

Output:

Run 1:

```

-----
Enter day number: 4↓
WEDNESDAY.

```

Run 2:

```

-----
Enter day number: 2↓
MONDAY.

```

Run 3:

```

-----
Enter day number: 10↓
INVALID DAY.

```

C program to find largest number from four given numbers.

```

#include<stdio.h>

int main()
{
    float a,b,c,d, lg;

```



```
printf("Enter four numbers:\n");  
  
scanf("%f%f%f%f", &a, &b, &c, &d);
```

```
  
if(a>b && a>c && a>d)  
{  
    lg = a;  
}  
  
else if(b>a && b>c && b>d)  
{  
    lg = b;  
}  
  
else if(c>a && c>b && c>d)  
{  
    lg = c;  
}  
  
else  
{  
    lg = d;  
}  
  
printf("Largest = %f", lg);  
  
return(0);  
  
}
```

Output:

Run 1:

Enter four numbers:

21.4↓

54.4↓

34.5↓

-67.5↓

Largest = 54.400000

Note: ↓ indicates enter is pressed

Switch statement:

When there are several options and we have to choose only one option from the available ones, we can use switch statement. Depending on the selected option, a Particular task can be performed. A task represents one or more statements.

Syntax:

switch (expression)

{

case value-1:

statement/block-1;

break;

case value-2:

statement/block t-2;

break;

case value-3:

statement/block -3;

break;

case value-4:

statement/block -4;

```
break;
```

```
default:
```

```
default- statement/block t;
```

```
break;
```

```
}
```

The expression following the keyword switch in any „C“ expression that must yield an integer value. It must be an integer constants like 1, 2, and 3.

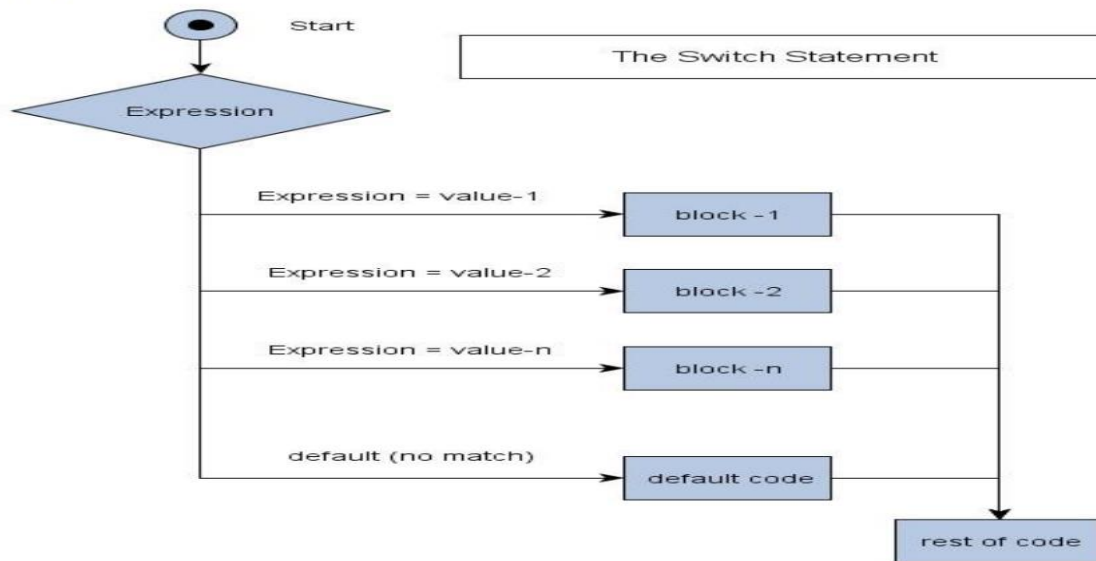
The keyword case is followed by an integer or a character constant, each constant in each must be different from all the other. First the integer expression following the keyword switch is evaluated. The value it gives is searched against the constant values that follow the case statements. When a match is found, the program executes the statements following the case. If no match is found with any of the case statements, then the statements following the default are executed.

Rules for writing switch () statement.

- 1: The expression in switch statement must be an integer value or a character constant.
- 2: No real numbers are used in an expression.
- 3: The default is optional and can be placed anywhere, but usually placed at end.
- 4: The case keyword must terminate with colon (:).
- 5: No two case constants are identical.
- 6: The case labels must be constants.

Valid Switch	Invalid Switch	Valid Case	Invalid Case
switch(x)	switch(f)	case 3;	case 2.5;
switch(x>y)	switch(x+2.5)	case 'a';	case x;
switch(a+b-2)		case 1+2;	case x+2;
switch(func(x,y))		case 'x'>'y';	case 1,2,3;

Flowchart



The working of the switch statement in C is as follows:

Step 1: The switch variable is evaluated.

Step 2: The evaluated value is matched against all the present cases.

Step 3A: If the matching case value is found, the associated code is executed.

Step 3B: If the matching code is not found, then the default case is executed if present.

Step 4A: If the break keyword is present in the case, then program control breaks out of the switch statement.

Step 4B: If the break keyword is not present, then all the cases after the matching case are executed.

Step 5: Statements after the switch statement are executed.

Example:

```
#include<stdio.h>

main()
{
    int a;
    printf("Please enter a no between 1 and 5: ");
    scanf("%d",&a);
    switch(a)
    {
        case 1:
            printf("You chose One");
```

```
break;

case 2:

printf("You chose Two");

break;

case 3:

printf("You chose Three");

break;

case 4:

printf("You chose Four");


break;

case 5: printf("You chose Five.");

break;

default :

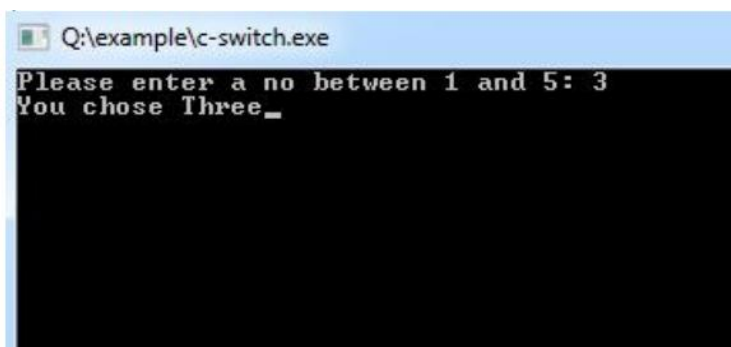
printf("Invalid Choice. Enter a no between 1 and 5");

break;

}

}
```

OUTPUT:



Points to Remember

It isn't necessary to use break after each block, but if you do not use it, all the consecutive block of codes will get executed after the matching block

```
1. int i = 1;
2. switch(i)
3. {
4.     case 1:
5.         printf("A");           // No break
6.     case 2:
7.         printf("B");           // No break
8.     case 3:
9.         printf("C");
10.    break;
11. }
```

Output: A B C

The output was supposed to be only A because only the first case matches, but as there is no break statement after the block, the next blocks are executed, until the cursor encounters a break.

default case can be placed anywhere in the switch case. Even if we don't include the default case switch statement works.

C program to print weekday based on given number using switch

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int day;
```

```
    printf("Enter day number: ");
```

```
    scanf("%d", &day);
```

```
    switch(day)
```

```
    {
```

```
        case 1: printf("SUNDAY.");
```

```
            break;
```

```
        case 2: printf("MONDAY.");
```

```
            break;
```

```

        case 3: printf("TUESDAY.");

                break;

        case 4: printf("WEDNESDAY.");

                break;

        case 5: printf("THURSDAY.");

                break;

        case 6: printf("FRIDAY.");

                break;

        case 7: printf("SATURDAY.");

                break;

        default: printf("INVALID DAY.");

                break;

    }

    return(0);

}

```

Output:

run 1:

Enter day number: 3↓

TUESDAY.

Run 2:

Enter day number: -3↓

INVALID DAY.

C program to implement simple calculator using switch statement

```
#include<stdio.h>
```

```
int main()
```

```

{

    float a,b, r;

    char op;

    printf("Available Operations.\n");

    printf("+ for Addition.\n");

    printf("- for Subtraction.\n");

    printf("* for Multiplication.\n");

    printf("/ for Division.\n");

    printf("Which Operation?\n");

    scanf("%c", &op);

    switch(op)
    {

        case '+': printf("Enter two numbers:\n");

                scanf("%f%f",&a, &b);

                r = a+b;

                printf("%f + %f = %f", a, b, r);

                break;

        case '-': printf("Enter two numbers:\n");

                scanf("%f%f",&a, &b);

                r = a-b;

                printf("%f - %f = %f", a, b, r);

                break;

        case '*': printf("Enter two numbers:\n");

                scanf("%f%f",&a, &b);

                r = a*b;

```



```

        printf("%f * %f = %f", a, b, r);

        break;

    case '/': printf("Enter two numbers:\n");

        scanf("%f%f",&a, &b);

        if(b!=0)

        {

            r = a/b;

            printf("%f/%f = %f",a,b,r);

        }

        else

        {

            printf("Division not possible.");

        }

        break;

    default: printf("Invalid Operation.");

        break;

}

return(0);

}

```

Output:

Run 1:

Available Operations.

+ for Addition.

- for Subtraction.

* for Multiplication.

/ for Division.

Which Operation?

* ↵

Enter two numbers:

13↵

17↵

13 * 17 = 221

Run 2:

Available Operations.

+ for Addition.

- for Subtraction.

* for Multiplication.

/ for Division.

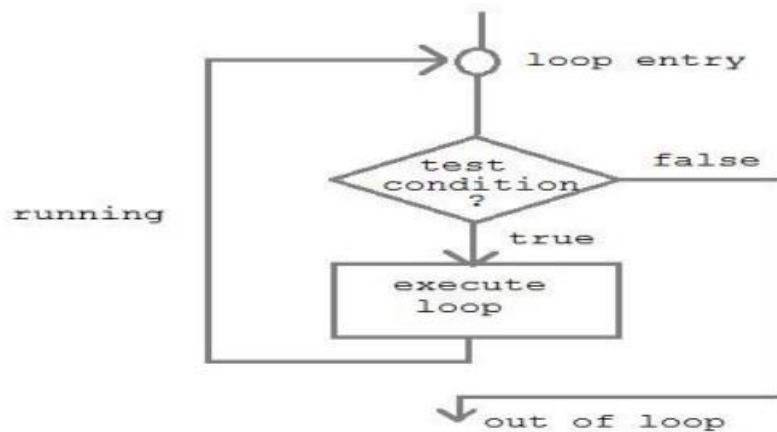
Which Operation?

^↵

Invalid Operation.

Iteration Statements/ Loop Control Statements:

How it Works



A sequence of statements are executed until a specified condition is true. This sequence of statements to be executed is kept inside the curly braces { } known as the Loop body. After every execution of loop body, condition is verified, and if it is found to be true the loop body is executed again. When the condition check returns false, the loop body is not executed.

The loops in C language are used to execute a block of code or a part of the program several times. In other words, it iterates/repeat a code or group of code many times.

Or

Looping means a group of statements are executed repeatedly, until some logical condition is satisfied.

Why use loops in C language?

Suppose that you have to print table of 2, then you need to write 10 lines of code. By using the

Loop statement, you can do it by 2 or 3 lines of code only.

A looping process would include the following four steps.

- 1: Initialization of a condition variable.
- 2: Test the condition.
- 3: Executing the body of the loop depending on the condition.
- 4: Updating the condition variable

C language provides three iterative/repetitive loops.

- 1: for loop
- 2: while loop

3: do-while loop

1: for loop:

The for loop is most commonly used looping statement. It is pre-tested loop and it is used when the number of repetitions (iterations) are known in advance. The syntax of the loop consists three parts, they are, control variable initialization, condition and control variable update. These three parts are separated by semicolon.

Syntax: for (control_variable_initialization; condition; control_variable_update)

```
{  
    statement(s);    //body of loop  
}
```

Note: Curly braces are optional if body of loop consists single statement.

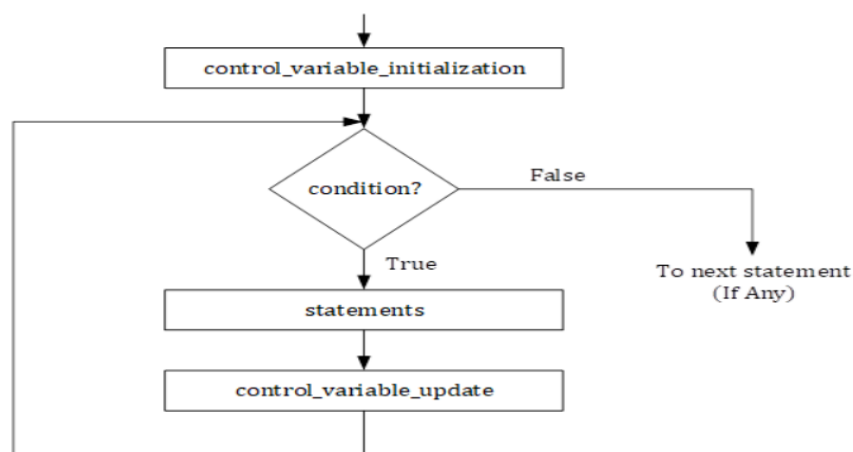
for Loop Working: In for loop control variable is initialized first. After initialization of control variable, it tests condition. Based on the outcome of condition, loop is continued or stopped.

If the condition evaluates to FALSE, then loop will be terminated (goes to next statement following loop).

If the condition evaluates to TRUE, then statements within body of loop are executed.

After execution of statements, value of loop control variable is updated and condition is checked again. And the process is repeated.

for Loop Flowchart



Example: Program to print “Programming is fun.” 10 times

```
#include<stdio.h>

int main()

{

    int i;

    for(i=1;i<=10; i++)

    {

        printf("Programming is fun.\n");

    }

    return(0);

}
```

Output:

Programming is fun.

Programming is fun.

Programming is fun.

Programming is fun.

Programming is fun.

Programming is fun.

Programming is fun.

Programming is fun.

Programming is fun.

Programming is fun.

Program to print “Programming is fun.” n times, where n is given by user

```
#include<stdio.h>
```

```
int main()

{

    int i, n;

    printf("Enter n: ");

    scanf("%d", &n);

    for(i=1;i<=n; i++)

    {

        printf("Programming is fun.\n");

    }

    return(0);

}
```

Output

Enter n: 5 ↵

Programming is fun.

Programming is fun.

Programming is fun.

Programming is fun.

Programming is fun.

Note: ↵ indicates enter is pressed.

While Loop:

Syntax:

Variable initialization;

While (condition)

```
{
```

Statements;

Variable increment or decrement;

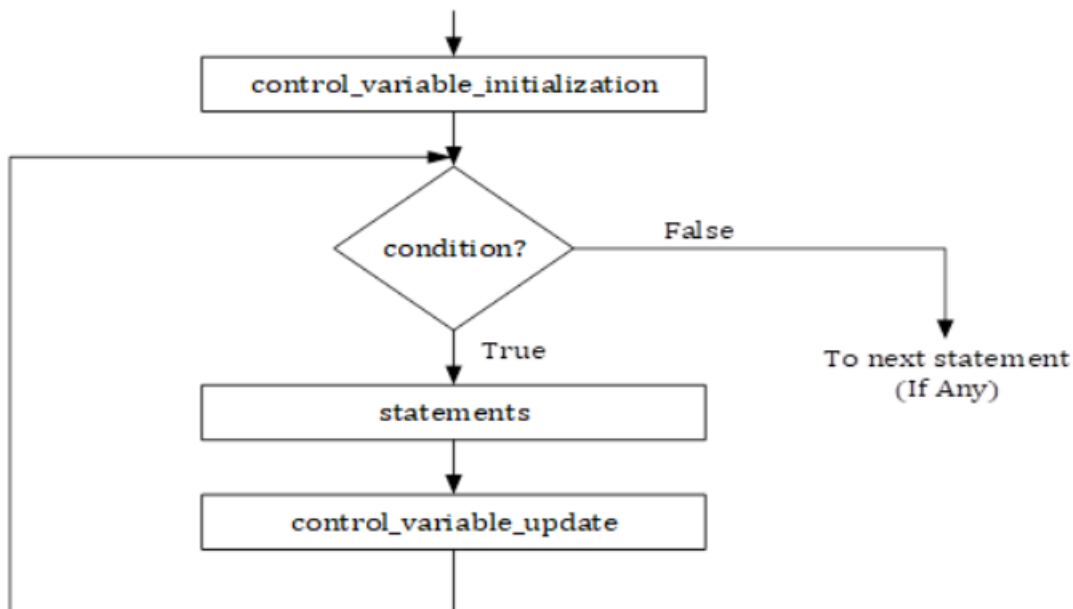
}

while loop can be addressed as an entry control loop. It is completed in 3 steps.

- Variable initialization.(e.g int x=0;)
-
- condition(e.g while(x<=10))
- Variable increment or decrement (x++ or x-- or x=x+2)

Working: The while loop is an entry controlled loop statement, i.e means the condition is evaluated first and it is true, then the body of the loop is executed. After executing the body of the loop, the condition is once again evaluated and if it is true, the body is executed once again, the process of repeated execution of the loop continues until the condition finally becomes false and the control is transferred out of the loop.

while Loop Flowchart



Example #1: Program to display 1 to 10 using while loop.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()

{

    int i;


    clrscr();

    i=1;

    while(i<=10)

    {

        printf("%d\n",i);

        i++;

    }

    getch();

}
```

Output

```
1
2
3
4
5
6
7
8
9
10
```

Example #2: Program to find sum of digit of a given number.


```
#include<stdio.h>

#include<conio.h>

void main()

{

    int num, rem, sum=0;

    clrscr();

    printf("Enter number: ");

    scanf("%d", &num);

    while(num!=0)

    {

        rem = num%10;

        sum = sum + rem;

        num = num/10;

    }

    printf("Sum is %d", sum);

    getch();

}
```

Output

Run1:

Enter number: 15234↓

Sum is 15

----- Run2:

Enter number: 6878↓

Sum is 29

Example #3: Program to reverse a given number.

```
#include<stdio.h>

#include<conio.h>

void main()

{

    int num, rem, rev=0;

    clrscr();

    printf("Enter number: ");

    scanf("%d", &num);

    while(num!=0)

    {

        rem = num%10;

        rev = rev*10 + rem;

        num = num/10;

    }

    printf("Reverse is %d", rev);

    getch();

}
```

Output

Run1:

Enter number: 1234↵

Reverse is 4321

do-while:

Syntax

```
control_variable_initialization;
```

```
do
```

```
{
```

```
    Statement;
```

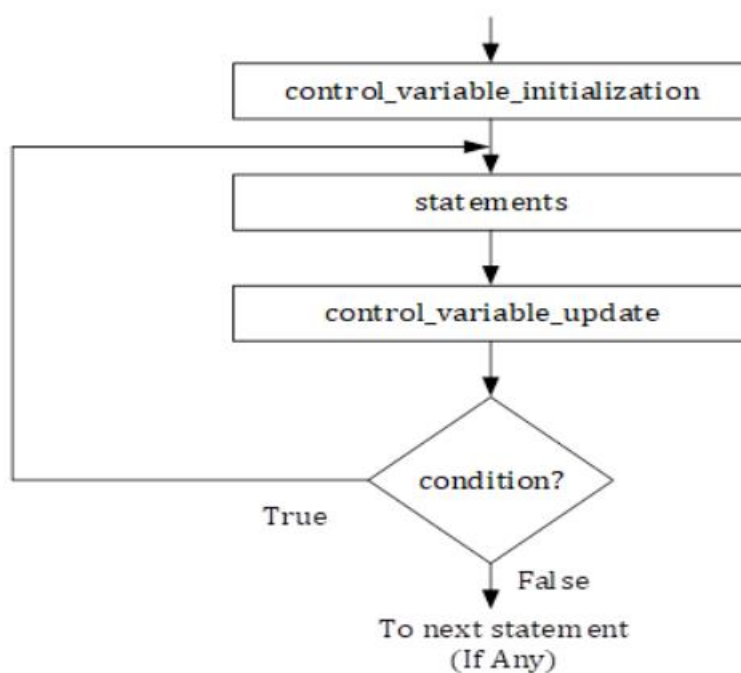
```
    control_variable_update;
```

```
} while (condition);
```

Working:

The do-while loop is an exit controlled loop statement the body of the loop are executed first and then the condition is evaluated. If it is true, then the body of the loop is executed once again. The process of execution of body of the loop is continued until the condition finally becomes false and the control is transferred to the statement immediately after the loop. The statements are always executed at least once.

do while Loop Flowchart



There is given the simple program of c language do while loop where we are printing the table of 1

```
#include<stdio.h>
```

```
int main(){
```

```
int i=1;
```

```
do{
```

```
printf("%d \n",i);
```

```
i++;
```

```
}while(i<=10);
```

```
return 0;
```

```
}
```

Output:

1

2

3

4

5

6

7

8

9

10

Program to print table for the given number using do while loop

```
#include<stdio.h>
```

```
int main(){
```

```
int i=1,number=0;
```

```
printf("Enter a number: ");  
  
scanf("%d",&number);  
  
do{  
  
printf("%d \n",(number*i));  
  
i++;  
  
}while(i<=10);  
  
return 0;  
  
}
```

Output

Enter a number: 5

5

10

15

20

25

30

35

40

45

50

Example 3: C program to check whether a given number is Palindrome or not using do-while.

```
#include<stdio.h>  
  
#include<conio.h>  
  
void main()  
  
{  
  
int num, rem, rev=0, copy;
```

```
clrscr();

printf("Enter number: ");

scanf("%d", &num);

copy = num;

do

{

    rem = num%10;

    rev = rev*10 + rem;

    num = num/10;

} while(num!=0);


if(rev==copy)

{

    printf("PALINDROME");

}

else

{

    printf("NOT PALINDROME");

}

getch();

}
```

Output:

Run1:

Enter number: 1234↵

NOT PALINDROME

Run2:

Enter number: 1221↵

PALINDROME

Unconditional Control Statements

In c, there are control statements that do not need any condition to control the program execution flow. These control statements are called as unconditional control statements. C programming language provides the following unconditional control statements...

break

continue

goto

The above three statements do not need any condition to control the program execution flow.

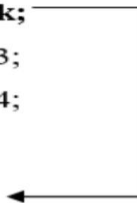
The break statement ends the loop immediately when it is encountered. The break statement is used to perform the following two things...

1. break statement is used to terminate the switch case statement
2. break statement is also used to terminate looping statements like while, do-while and for.

The syntax for break statement is as follows –

Syntax

```
{  
    Stmt1;  
    Stmt2;  
    break;  
    Stmt3;  
    Stmt4;  
}
```



When a **break** statement is encountered inside the switch case statement, the execution control moves out of the switch statement directly. For example, consider the following program

Example: C program to print weekday based on given number using switch

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int day;
```

```
    printf("Enter day number: ");
```

```
scanf("%d", &day);

switch(day)

{

    case 1: printf("SUNDAY.");

        break;

    case 2: printf("MONDAY.");

        break;

    case 3: printf("TUESDAY.");

        break;

    case 4: printf("WEDNESDAY.");

        break;

    case 5: printf("THURSDAY.");

        break;

    case 6: printf("FRIDAY.");

        break;

    case 7: printf("SATURDAY.");

        break;

    default: printf("INVALID DAY.");

        break;

}

return(0);

}
```

Output:

run 1:

Enter day number: 3↵

TUESDAY.

Run 2:

Enter day number: -3↵

INVALID DAY.

When the break statement is encountered inside the looping statement, the execution control moves out of the looping statements. The break statement execution is as shown in the following figure.

.Example:

```
#include<stdio.h>

main( )
{
    int i;

    for (i=1; i<=5; i++){

        printf ("%d", i);

        if (i==3)

            break;

    }

}
```

Output

1 2 3

continue: The `continue` statement skips the current iteration of the loop and continues with the next iteration

The syntax for the continue statement is as follows –

Syntax

```
{  
    Stmt1;  
    Stmt2;  
    continue;   
    Stmt3;  
    Stmt4;  
}
```

Example

Following is the C program for the continue statement –

```
#include<stdio.h>  
  
main( ){  
    int i;  
    for (i=1; i<=5; i++){  
        if (i==2)  
            continue;  
        printf("%d", i)  
    }  
}
```

Output

1 2 3 4 5

You can also use **break** and **continue** in while loops:

```
int i = 0;  
  
while (i < 10)
```

```
{  
    if (i == 4)  
    {  
        break;  
    }  
    printf("%d\n", i);  
    i++;  
}
```

Output:

0
1
2
3

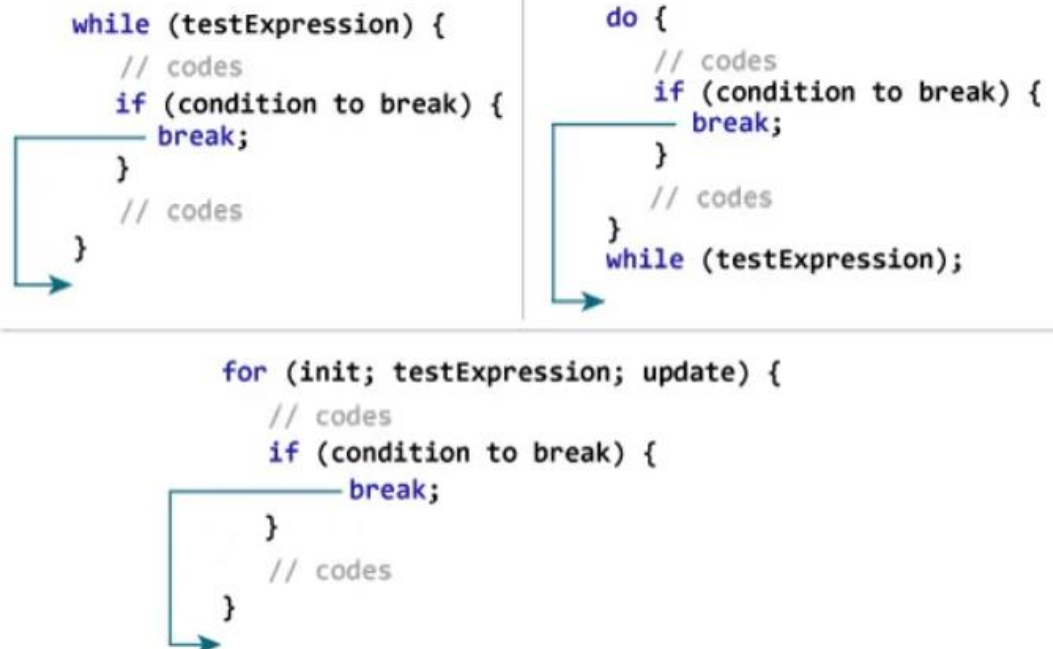
Continue example:

```
int i = 0;  
while (i < 10)  
{  
    if (i == 4)  
    {  
        i++;  
        continue;  
    }  
    printf("%d", i);  
    i++;  
}
```

Output:

0 1 2 3 5 6 7 8 9

How break statement works?



Working of break in C

How continue statement works?



Working of Continue in C

goto Statement:

The goto statement allows us to transfer control of the program to the specified label.

Syntax of goto Statement

```
goto label;
```

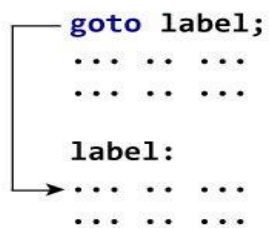
```
... ..
```

```
... ..
```

```
label:
```

```
statement;
```

The label is an identifier. When the goto statement is encountered, the control of the program jumps to label: and starts executing the code.



Example:

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    int age;
```

```
    g: //label name
```

```
        printf("you are Eligible\n");
```

```
    s: //label name
```

```
        printf("you are not Eligible");
```

```
    printf("Enter you age:");
```

```
    scanf("%d", &age);
```

```
    if(age>=18)
```

```
goto g; //goto label g
```

```
else
```

```
goto s; //goto label s
```

```
}
```

Output:

Enter your age: 19

you are Eligible
